

UNIVERSIDADE FEDERAL DO CEARÁ
NOME DO CENTRO, FACULDADE, INSTITUTO OU CAMPUS
CURSO DE GRADUAÇÃO EM ENGENHARIA DE COMPUTAÇÃO

EDUARDO SERPA

VALIDAÇÃO AUTOMATIZADA DE DOCUMENTOS ACADÊMICOS EM LATEX: UM ESTUDO DEMONSTRATIVO

FORTALEZA
2026

EDUARDO SERPA

VALIDAÇÃO AUTOMATIZADA DE DOCUMENTOS ACADÊMICOS EM LATEX: UM ESTUDO DEMONSTRATIVO

Trabalho de Conclusão de Curso apresentado ao Curso de Graduação em Engenharia de Computação da Universidade Federal do Ceará, como requisito parcial à obtenção do grau de bacharel em Engenharia de Computação.
Orientador: Prof. Dr. Paulo Antonio Leal Rego.

FORTALEZA

2026

EDUARDO SERPA

VALIDAÇÃO AUTOMATIZADA DE DOCUMENTOS ACADÊMICOS EM LATEX: UM ESTUDO DEMONSTRATIVO

Trabalho de Conclusão de Curso apresentado ao Curso de Graduação em Engenharia de Computação da Universidade Federal do Ceará, como requisito parcial à obtenção do grau de bacharel em Engenharia de Computação.

Aprovada em: 12 de setembro de 2026

BANCA EXAMINADORA

Prof. Dr. Paulo Antonio Leal Rego
Universidade Federal do Ceará (UFC)

Profa. Dra. Nome do Segundo Membro
Universidade Federal do Ceará (UFC)

Prof. Dr. Nome do Terceiro Membro
Universidade Federal do Ceará (UFC)

RESUMO

Este trabalho apresenta um fluxo demonstrativo para validação automatizada de documentos acadêmicos produzidos em LaTeX. O objetivo é mostrar, em um TCC curto e estruturalmente realista, como separar a preparação do conteúdo, a compilação do projeto, a execução de verificações automáticas e a revisão visual do arquivo final. O procedimento considera propriedades objetivas que podem ser observadas no PDF, como tamanho da página, incorporação de fontes, conformidade com o alvo PDF/A adotado e presença de marcadores estruturais. Para ilustrar o registro dos resultados, utiliza-se uma execução sintética com quatro verificações e uma métrica simples de taxa de aprovação. Os dados apresentados têm finalidade exclusivamente didática e não representam um experimento externo. O trabalho também demonstra a organização prática do projeto: metadados concentrados no arquivo principal, capítulos separados, bibliografia estruturada e uso contextual de figura, tabela, equação e código-fonte. Os resultados reforçam que a automação pode antecipar problemas repetitivos e tornar a verificação mais reprodutível, mas não substitui a avaliação do conteúdo científico, da metodologia, das fontes e das exigências específicas do curso ou programa.

Palavras-chave: LaTeX; trabalhos acadêmicos; validação; automação; normalização.

ABSTRACT

This work presents a demonstrative workflow for automated validation of academic documents produced in LaTeX. The objective is to show, through a short and structurally realistic undergraduate thesis, how to separate content preparation, project compilation, automated checks, and visual review of the final file. The procedure considers objective properties that can be observed in the PDF, including page size, font embedding, conformance with the adopted PDF/A target, and the presence of structural markers. To illustrate result reporting, the study uses a synthetic execution with four checks and a simple pass-rate metric. The presented data are exclusively didactic and do not represent an external experiment. The work also demonstrates practical project organization: metadata concentrated in the main file, separate chapter files, a structured bibliography, and contextual use of a figure, a table, an equation, and source code. The results emphasize that automation can identify repetitive problems earlier and make verification more reproducible, but it does not replace the evaluation of scientific content, methodology, consulted sources, or requirements specific to the applicable course or graduate program.

Keywords: LaTeX; academic works; validation; automation; standardization.

LISTA DE ILUSTRAÇÕES

LISTA DE TABELAS

LISTA DE CÓDIGOS

SUMÁRIO

1 INTRODUÇÃO

Com o advento dos smartphones e a expansão do acesso à internet móvel, os aplicativos de mensageria, como WhatsApp, Telegram, Discord e Slack, se tornaram parte fundamental da vida em sociedade em diversas esferas das vidas pública e privada. Frequentemente, esses serviços são mais baratos e mais flexíveis que os meios de comunicação que eles suplantaram, como ligações telefônicas e mensagens curtas de texto (SMS), que são sujeitas a cobrança pelas operadoras de telefonia.

Apesar dessas vantagens, há certos riscos relacionados ao caráter centralizado desses serviços de comunicação. Eles frequentemente são controlados por empresas privadas que operam internacionalmente, gerando maior incerteza quanto às leis de segurança digital que são aplicadas em diversos territórios e preocupações com a possibilidade de violações de privacidade e soberania nacional.

Uma das possíveis soluções é a substituição desses serviços centralizados por outros, baseados em paradigmas de computação distribuída, como o protocolo Matrix. Ele consiste em uma série de especificações em código aberto que padronizam a comunicação entre diversos sistemas independentes por meio de uma interface de programação de aplicações (API) que reúne microsserviços. Isso não só o torna compatível com outros serviços de mensageria como dá a grupos de usuários todo o espectro de opções entre ter um único provedor central e ter um servidor autônomo para cada usuário, compartilhando todas as informações relevantes para o grupo entre si.

Essa diferença de arquitetura traz certas vantagens, como maior controle sobre as políticas de privacidade, sobre as funções disponíveis e sobre as regras de uso às quais os usuários estão submetidos. Ela traz, porém, desafios técnicos para o funcionamento da aplicação, pois a ausência de um servidor central implica que a carga de trabalho total é dividida entre múltiplos nós da rede, incluindo os usuários. Torna-se, então, necessária a existência de algoritmos e fluxos de informação capazes de produzir um consenso entre os nós da rede.

Para os serviços de mensageria baseados no padrão Matrix, o consenso é produzido a partir do processo de sincronização, que replica informações entre os provedores de serviço, garantindo que estejam disponíveis e consistentes para todos os usuários que compartilham espaços e que devem, portanto, ser capazes de acessar aquelas informações. Visando compensar os custos de processamento e de transferência de dados, a versão 2.0 do protocolo Matrix propõe o mecanismo de Simplified Sliding Sync, que se diferencia do endpoint /sync tradicional por [explicação, comparação].

Dentre as implementações de mensageria com o padrão Matrix, o Synapse se destaca por ter sido a primeira e mais frequentemente atualizada, servindo de referência para as demais. A versão 1.114.0, lançada em setembro de 2024, é um marco importante por ter sido a primeira a utilizar nativamente o padrão 2.0 (). Comparando o desempenho dessa versão com a anterior, é possível quantificar os ganhos imediatos com essa alteração, mas é, também, necessário comparar com a versão mais recente para uma avaliação mais completa, incluindo o amadurecimento do Simplified Sliding Sync.

[Objetivos, relação com a problemática e possíveis ganhos/justificativa]

Métricas úteis para esse fim podem ser obtidas a partir de simulações em contêineres orquestrados por Docker Compose. A stack de simulação consiste, principalmente, em: uma instância de servidor Matrix com workers especializados para diferentes microsserviços (Synapse, implementado na linguagem Python), uma instância de um serviço que simula as requisições de usuário comuns (Locust, modificado para interagir com a API no padrão Matrix), e um proxy reverso (Nginx), que faz a ponte entre os dois para distribuir a carga entre workers especializados e facilitar a observabilidade entre diferentes funções da API.

Essa configuração de componentes será

1.1 Problema

Descrição do problema.

1.2 Objetivos

Quantificar o impacto da implementação de Simplified Sliding Sync sobre o desempenho da sincronização de dados por nós de uma rede federada Matrix baseados em Synapse.

a) Estabelecer como padrão um ambiente de testes facilmente reprodutível, composto por ferramentas de conteinerização e de automação (Docker Compose e Bash), stack de simulação (Synapse Matrix, Locust e Nginx) e stack de telemetria (OpenTelemetry, Prometheus, Jaeger, Grafana);
b) Formatar e utilizar dados reais de uso de aplicativos de mensageria para orientar as simulações realizadas com o Locust;
c) Realizar comparações estatísticas de desempenho entre três versões de Synapse Matrix: a última antes da implementação nativa de Simplified Sliding Sync, a

primeira com esse mecanismo e, por fim, a mais recente.

2 FUNDAMENTAÇÃO TEÓRICA

A problemática e a definição do experimento dependem de conceitos de computação e de redes de computadores para serem explicados.

2.1 Computação distribuída

Tanenbaum e Steen (2007c) definem o sistema distribuído como 'um conjunto de computadores independentes que se apresenta a seus usuários como um sistema único e coerente'. Por meio da internet, esse conjunto pode ser composto por entidades em diversas localidades e sob diversos domínios administrativos, desde que compartilhem uma interface que define o comportamento de cada uma em relação às outras e aos usuários. Essa camada intermediária que abstrai a heterogeneidade e a complexidade da rede é chamada de middleware.

Os nós de um sistema distribuído podem se relacionar de diversas formas, dependendo, por exemplo, do domínio administrativo sob o qual eles operam. Sistemas de arquitetura centralizada são caracterizadas pela relação entre servidores, que proveem um serviço, e clientes, que requisitam esses serviços (Tanenbaum; Steen, 2007a). É o caso de serviços de mensageria instantâneos como Whatsapp e Discord, nos quais uma empresa intermedia toda a comunicação entre usuários, que passa criptografada por seus servidores para ser requisitada e descriptografada pelo cliente do usuário na outra ponta (citation-needed-2).

Por outro lado, os sistemas de arquitetura federada são caracterizados pela cooperação entre nós completamente independentes, mas que compartilham as responsabilidades de gerenciar e de armazenar informações. Na prática, o arranjo mais comum é uma abordagem híbrida, na qual conjuntos de usuários têm relações centralizadas em servidores específicos que interagem entre si de modo completamente independente (Tanenbaum; Steen, 2007b). É o caso, por exemplo, de serviços de mensageria instantânea baseados em padrões abertos como XMPP e Matrix (citation-needed-2). De modo análogo aos provedores de serviço de e-mail, cada *homeserver* guarda os dados de seus usuários e intermedia a comunicação com outros domínios e seus respectivos usuários.

Essa descentralização pode trazer importantes vantagens, como mais privacidade, resiliência, tolerância a falhas, escalabilidade e independência de provedores privados. Há, porém, custos operacionais decorrentes das arquiteturas distribuídas. Sem a autoridade de um nó central, é necessário executar algoritmos de consenso para manter a consistência e a integridade dos dados, gerando maior volume de comunicação na rede. Qualquer operação que afete nós

afiliados a diferentes homeservers resulta, então, em maior consumo de recursos para transporte, armazenamento e processamento de dados, causando latência e perda de desempenho com o processo de sincronização (citation-needed-1).

2.2 Serviços de mensageria instantânea

São, em resumo, sistemas de comunicação assíncrona e intermediada por um provedor de serviço (denominado 'intermediário' ou *message broker*), que recebe mensagens de um remetente ('produtor' ou *publisher*), notifica o destinatário ('consumidor' ou *subscriber*) e as repassa a ele quando forem requisitadas por este (citation-needed-1). As opções mais populares, como SMS, WhatsApp, Telegram e Discord (citation-needed-2), utilizam arquiteturas centralizadas em servidores de propriedade das empresas que proveem esse serviço, facilitando sua adoção pelos usuários, que precisam apenas instalar o cliente local em um de seus dispositivos (**shipley-bowker-communication**).

Há, porém, preocupações quanto aos riscos associados a esse enorme fluxo de dados entre pessoas e empresas estrangeiras. Embora não funcionem como fóruns abertos, os serviços de mensageria estão profundamente integrados às dinâmicas sociais e econômicas de grande parte da população (Faria; Silveira, 2023), ampliando o possível impacto de interrupções de serviço, de alterações nos Termos de Uso da aplicação ou de uso indevido de informações e canais de comunicação por empresas ou governos. Para mitigar esses riscos, organizações e pessoas podem adotar ou desenvolver alternativas baseadas em arquitetura federada. Dentre os principais exemplos, estão diversos órgãos de governos europeus, como França, Alemanha, Suíça e Reino Unido (Soluções [. . .], s.d.).

2.3 Matrix e Synapse

O protocolo Matrix é um padrão aberto, baseado em princípios de computação distribuída, que possibilita a comunicação entre nós de uma rede de arquitetura federada. Ele é mantido pela Matrix.org Foundation, que se propõe a ser um agente neutro e aberto sobre sua evolução, estabelecendo um conjunto mínimo de interfaces que torna diferentes implementações do protocolo interoperáveis (About [. . .], s.d.).

Dentre as implementações baseadas em Matrix, o Synapse se destaca por ter sido o primeiro, desenvolvido pelos próprios criadores do protocolo, e por ser o mais atualizado. Esses fatores contribuem para a popularidade de seu cliente associado, o Element, e para sua própria

como *homeserver*, além de servir como referência para outras implementações (The [. . .], s.d.).

A principal unidade de informação do Matrix é a sala, que reúne usuários oriundos de quaisquer *homeservers* para compartilhar um conjunto de informações que, sendo comum a todos eles, são armazenadas como cópias criptografadas por cada um dos provedores envolvidos na comunicação. Além disso, os usuários podem facilmente transferir seus dados para outro servidor. Essa redundância garante que a disponibilidade dos dados só seja ameaçada por uma falha simultânea de todos os *homeservers* (Matrix [. . .], s.d.).

O ecossistema Matrix permite, também, integração com outros serviços de mensageria por meio de pontes (ou "*bridges*") que traduzem protocolos e formatos de dados entre os sistemas, elimina a necessidade de dividir recursos e atenção entre múltiplas aplicações. Embora esse fator favoreça a adoção do protocolo por novos usuários, ele também traz riscos à privacidade, pois esse processo de tradução requer que a ponte decodifique a mensagem, implicando na perda da encriptação de ponta-a-ponta (ou *end-to-end encryption*, E2EE). É, portanto, recomendado que a ponte seja hospedada pelo próprio usuário ou por um provedor confiável (Matrix [. . .], s.d.).

2.4 Sincronização antes do Matrix 2.0

Todas as atualizações sobre a sala eram compartilhadas com todos os outros participantes daquela sala mediante requisição de sincronização, que é realizada pelos clientes locais dos usuários sempre que eles se conectam à rede e, periodicamente, a cada 30 segundos. Como as atualizações são replicadas em todos os clientes locais, basta que um deles esteja atualizado para que os demais também estejam (citation-needed-1).

Esse volume de troca de informações cresce drasticamente com a quantidade de usuários ativos e provedores envolvidos em uma mesma sala, gerando atrasos e gargalos de desempenho no processo de sincronização. Como forma de mitigação do problema, os servidores e clientes locais podem definir um intervalo de tempo para manter apenas as mensagens recentes (citation-needed-1).

2.5 Matrix 2.0 e Simplified Sliding Sync

Dentre outras mudanças propostas no projeto Matrix 2.0, a implementação do mecanismo de *Simplified Sliding Sync* se destaca por propor uma alternativa mais eficiente para o processo de sincronização. Consiste, essencialmente, em ignorar informações que não são imediatamente relevantes para o usuário que requisitou atualizações, como pequenas alterações

de estado entre mensagens de uma conversa. Desse modo, o contexto do usuário passa a orientar o movimento do intervalo de interesse do cliente local sobre o histórico de uma sala ao invés de repetir todos os passos necessários para sincronizar os estados (citation-needed-1).

É, portanto, necessário simular os padrões de comportamento reais para mensurar o impacto que o SSS tem sobre o desempenho, uma vez que essa otimização depende diretamente do contexto de uso da aplicação.

2.6 Containerização e automação

Para garantir a reprodutibilidade dos experimentos, as condições precisam ser controladas e isoladas das particularidades do sistema no qual ele é executado. As ferramentas mais comumente utilizadas para isso são máquinas virtuais e contêineres, com o segundo emulando apenas a camada de *software*, enquanto o primeiro reproduz, também, condições de *hardware*, precisando de mais recursos para realizar testes em escala semelhante (citation-needed-1).

Dentre as ferramentas de containerização, o Docker Compose se destaca pela facilidade de configurar, executar e documentar as dependências de um sistema composto por um conjunto de processos coordenados, mas atuando em contêineres próprios. Dentre esses processos, estão o servidor Synapse, com múltiplos *workers* especializados em diferentes tarefas necessárias para a sincronização (citation-needed-1).

3 METODOLOGIA

AAAAAAA

3.1 Configuração do ambiente de testes

3.1.1 Requisitos de software

AAAAAAA

- Git para clonar o repositório do projeto em github.com/Eduardoserpa/locust-stack-setup;
- Docker Compose para orquestração de containers;
- Interpretador de script Bash para automatizar a configuração do ambiente de testes;

3.2 Utilizando os scripts

```
bash setup.sh
```

4 RESULTADOS E DISCUSSÃO

4.1 Interpretação

4.2 Uso do modelo

5 CONCLUSÃO

Este trabalho apresentou um fluxo demonstrativo para validação automatizada de documentos acadêmicos produzidos em LaTeX. A proposta separou preparação, compilação, verificações automáticas e revisão visual, mostrando como propriedades objetivas do PDF podem ser verificadas sem atribuir à automação decisões que continuam dependentes de análise humana.

O exemplo também demonstrou uma forma de organizar o projeto acadêmico: o arquivo principal concentra metadados e ordem dos elementos, enquanto capítulos, resumo, abstract, referências e materiais complementares permanecem em arquivos específicos. Figuras, tabelas, equações e código aparecem no contexto metodológico em que são utilizados.

Como limitação, os resultados apresentados são sintéticos e servem apenas para mostrar a estrutura do documento. Em um TCC real, objetivos, método, dados, resultados e conclusões devem decorrer da pesquisa efetivamente realizada. A documentação completa dos comandos da classe permanece separada deste texto para que o PDF do aluno continue sendo um trabalho acadêmico, e não um manual do template.

REFERÊNCIAS

ABOUT Matrix. [*S. l.: s. n.*].

FARIA, Marcelo Aparecido de; SILVEIRA, Sérgio Amadeu da. O WhatsApp e a Plataformização no Brasil: uma descrição densa dos agentes articulados nas práticas de controle mediadas pela plataforma. **Intercom: Revista Brasileira de Ciências da Comunicação**, SciELO Brasil, v. 46, e2023136, 2023. DOI: 10.1590/1809-58442023136pt. Disponível em: https://doi.org/10.1590/1809-58442023136pt.

MATRIX Bridging. [*S. l.: s. n.*].

MATRIX Rooms and Events. [*S. l.: s. n.*].

SOLUÇÕES baseadas em Matrix adotadas na Europa. [*S. l.: s. n.*].

TANENBAUM, Andrew S.; STEEN, Maarten van. *In:* TANENBAUM, Andrew S.; STEEN, Maarten van. **Sistemas Distribuídos**. Tradução: Arlete Simille Marques. 2. ed. São Paulo: Pearson Education, 2007a. cap. 2, p. 22–23. ISBN 9788576051428.

TANENBAUM, Andrew S.; STEEN, Maarten van. *In:* TANENBAUM, Andrew S.; STEEN, Maarten van. **Sistemas Distribuídos**. Tradução: Arlete Simille Marques. 2. ed. São Paulo: Pearson Education, 2007b. cap. 2, p. 31–32. ISBN 9788576051428.

TANENBAUM, Andrew S.; STEEN, Maarten van. **Sistemas Distribuídos**. Tradução: Arlete Simille Marques. 2. ed. São Paulo: Pearson Education, 2007c. p. 1–2, 22–23, 31–32. 402 p. ISBN 9788576051428.

THE relationship between Element and Matrix. [*S. l.: s. n.*].

GLOSSÁRIO

APÊNDICE A — CHECKLIST DE VALIDAÇÃO USADO NO ESTUDO DEMONSTRATIVO

Este apêndice exemplifica material elaborado pelo próprio autor. No estudo demonstrativo, o checklist mínimo utilizado foi:

1. confirmar que o documento foi compilado sem erros;
2. verificar o tamanho A4 das páginas;
3. verificar a incorporação das fontes utilizadas;
4. executar a validação PDF/A disponível no ambiente;
5. confirmar a presença dos principais elementos estruturais;
6. revisar visualmente o PDF completo.

Em um trabalho real, substitua este conteúdo por material autoral complementar que seja pertinente à pesquisa.

ANEXO A — DOCUMENTO COMPLEMENTAR EXTERNO

Este anexo demonstra a posição reservada para um documento complementar que não tenha sido elaborado pelo autor do trabalho.

Substitua este texto pelo material externo efetivamente necessário à pesquisa. Verifique previamente direitos de uso, legibilidade e compatibilidade com o arquivo final.

Fonte: Instituição ou autor responsável pelo documento externo (ano). Substitua esta indicação pela fonte real do material anexado.